

Τεχνολογίες Αποκεντρωμένων Δεδομένων 2023-2024

Project2_1: Μελέτη-Υλοποίηση DHT με Εφαρμογές του

ΛΕΥΤΕΡΗΣ ΑΜΙΤΣΗΣ 1072464
ΣΑΡΑΝΤΗΣ ΠΑΠΑΧΡΙΣΤΟΦΙΛΟΥ 1072600

Για την υλοποίηση του πρότζεκτ θα γίνει περιγραφή του προγράμματος και ύστερα η πειραματική αξιολόγηση μαζί με τα σχετικά screenshots απο τον κώδικα.

Υλοποίηση Chord Circle

Βιβλιοθήκες

Οι βιβλιοθήκες που χρησιμοποιήθηκαν αρχικά είναι τα requests (responses) για την πρόσβαση μας στην σελίδα της Wikipedia με του επιστήμονες και η BeautifulSoup για την δημιουργία του web crawler. Έπειτα χρησιμοποιήθηκε η csv για εισαγωγή του dataset σε ένα csv για την εξοικονόμηση χρόνου. Επίσης χρησιμοποιήθηκε η sys για εντολές συστήματος. Τέλος οι εντολές math για αριθμητικές πράξεις και η pandas για δημιουργία dataframes.

```
import requests
from bs4 import BeautifulSoup
import csv
import pandas as pd
import math
import sys
```

Web crawler

Για τις ανάγκες αυτού του project μας ζητήθηκε να δημιουργήσουμε ένα web crawler που θα εξάγει δεδομένα απο το παρακάτω url και θα εξάγει τις πληροφορίες των επιστημόνων που περιέχει.

```
url = "https://en.wikipedia.org/wiki/List_of_computer_scientists"
```

Με την χρήση της βιβλιοθήκης beautiful soup και την βοήθεια των dev tools του mozilla δημιουργήσαμε ένα κομμάτι κώδικα που θα παίρνει ένα ένα τα links που συνδέονται με τα ονόματα των επιστημόνων, μπαίνει στην σελίδα τους στο Wikipedia και εξάγει τις απαραίτητες πληροφορίες από το profile τους.

[illegible]

Όπως βλέπουμε παρακάτω κάνουμε request στην σελίδα και παίρνουμε ένα response για να συνδεθούμε. Έπειτα περιηγούμαστε στην html της σελίδας για να βρούμε τα links που μας ενδιαφέρουν. Έστερα δημιουργούμε μία ακόμα λίστα που έχει μέσα τα links που δεν χρειαζόμαστε αλλά αναγκαστικά ανακτήσαμε και τα αφαιρούμε από την τελική λίστα.

Links που αφαιρέσαμε:

See also [[edit](#)]

- List of computing people
- List of important publications in computer science
- List of Jewish American computer scientists
- List of members of the National Academy of Sciences (computer and information sciences)
- List of pioneers in computer science
- List of programmers
- List of programming language researchers
- List of Russian IT developers
- List of Slovenian computer scientists
- List of Indian computer scientists

References [[edit](#)]

External links [[edit](#)]

- CiteSeer list of the most cited authors in computer science
- Computer scientists with h-index >= 40

Wikimedia Commons has media related to **Computer scientists**.

Categories: [Lists of computer scientists](#) | [Computer scientists](#) | [Lists of people by occupation](#)

Κώδικας:

```
response = requests.get(url)
first_links_list = []

if response.status_code == 200:

    soup = BeautifulSoup(response.text, 'html.parser')

    h2_element = soup.find('div', {'class': 'mw-content-ltr mw-parser-output'})
    list_test=h2_element.find_all('ul')

    for ul_element in list_test:
        li_elements = ul_element.find_all('li')
        for li in li_elements:
            link = li.find('a')
            if link:
                href = link.get('href')
                if href:
                    full_link = f"https://en.wikipedia.org{href}"
                    first_links_list.append(full_link)

    links_to_exclude = []
    see_also_header = soup.find('span',id='See_also').parent
    see_also_div = see_also_header.find_next('div')
    see_also_ul = see_also_div.find_all('ul')
    for ul_element in see_also_ul:
        see_also_li_elements = ul_element.find_all('li')
        for li in see_also_li_elements:
            link = li.find('a')
            if link:
                href = link.get('href')
                if href:
                    full_link = f"https://en.wikipedia.org{href}"
                    links_to_exclude.append(full_link)
    first_links_list = [link for link in first_links_list if link not in links_to_exclude]
    print(first_links_list)
```

Έπειτα δημιουργήσαμε τρία functions για να εξάγουμε τις πληροφορίες του κάθε επιστήμονα από το προφίλ του εφόσον φυσικά υπάρχουν.

Ιδανικό προφίλ:

Patrick Henry Winston	
	
Born	February 5, 1943 Peoria, Illinois, U.S.
Died	July 19, 2019 (aged 76) Boston, Massachusetts, U.S.
Nationality	American
Alma mater	MIT (BS 1965, MS 1967, PhD 1970) ^[1]
Spouse	Karen Prendergast ^[2]
Children	1
Awards	IJCAI Computers and Thought Award

Όπως βλέπουμε στο προφίλ αυτό πάνω πάνω υπάρχει το όνομα, παρακάτω υπάρχουν τα βραβεία του και το alma mater (education) που ενδέχεται να υπάρχουν με άλλα ονόματα τα οποία συμπεριλήφθηκαν στον κώδικα. Ωστόσο υπάρχει περίπτωση κάποια προφίλ να μην περιέχουν όλες τις πληροφορίες που χρειαζόμαστε. Έτσι κάναμε την παραδοχή ότι σε όσους δεν υπάρχει το section awards θα έχουν μηδενικό αριθμό awards, σε όσους λείπει το όνομα θα βάζουμε unknown και σε όσους η εκπαίδευση (που είναι και το κλειδί άλλωστε) δεν θα συμπεριληφθούν .

Η υλοποίηση είναι παρόμοια με αυτή του πρώτου link. Κάνουμε request στην σελίδα παίρνουμε το response και μέσω της html της κάθε σελίδας παίρνουμε τις πληροφορίες που θέλουμε και τις κάνουμε return. Σε περίπτωση που δεν τις βρούμε κάνουμε return no save. Αυτή η διαδικασία πραγματοποιείται για όλα τα links στην λίστα που δημιουργήθηκε προηγουμένως.

Κώδικας:

Get name

```
1 def get_scientist_name(link):
2     response= requests.get(link)
3     if response.status_code == 200:
4
5         soup = BeautifulSoup(response.text, 'html.parser')
6         name=soup.find('span',{ 'class': 'mw-page-title-main'}).text
7         if name:
8             return name
9         else:
10            return 'unknown'
```

Get Awards

```
1 def get_scientist_awards(link):
2     response= requests.get(link)
3     if response.status_code == 200:
4
5         soup = BeautifulSoup(response.text, 'html.parser')
6         find_awards=0
7         awards_table = soup.find('table',{ 'class': 'infobox biography vcard'})
8         if(awards_table):
9             awards_table_condition = awards_table.find('th', string= 'Awards')
10            if awards_table_condition:
11                awards_table_td = awards_table_condition.find_next('td',{ 'class' : 'infobox-data'})
12                awards_table_list = awards_table_td.find_next('ul')
13                awards=len(awards_table_list.find_all('li'))
14                find_awards=1
15
16            if find_awards==0:
17                awards_header_condition = soup.find('span', { 'id': 'Honors_&_Awards'})
18                if awards_header_condition:
19                    awards_header = soup.find('span', { 'id': 'Honors_&_Awards'}).parent
20                    awards_list=awards_header.find_next('ul')
21                    li_elements = list(awards_list.find_all('li'))
22                    awards=len(li_elements)
23                    find_awards=1
24
25            if find_awards==1:
26                return awards
27            else:
28                return 0
```

Get education

```
1 def get_scientist_education(link):
2     response= requests.get(link)
3     if response.status_code == 200:
4
5         soup = BeautifulSoup(response.text, 'html.parser')
6         find_education=0
7         education_table = soup.find('table',{'class': 'infobox biography vcard'})
8         if education_table:
9             education_table_condition = education_table.find('th', string= 'Education')
10            if education_table_condition:
11                education_header= education_table_condition.parent
12                education_table_td = education_header.find_next('td')
13                education_list = education_table_td.find_next('> ul')
14                if education_list:
15                    education_text = ', '.join(li.get_text(strip=True) for li in ul_element.find_all('li'))
16                    find_education=1
17                education_a = education_table_td.find_all('a')
18                if education_a and find_education==0:
19                    education_text = ', '.join(a.get_text(strip=True) for a in education_table_td.find_all('a'))
20                    find_education=1
21            if find_education==0:
22                alma_matter_condition = education_table.find('th', string= 'Alma mater')
23                if alma_matter_condition:
24                    alma_matter_header = alma_matter_condition.parent
25                    alma_matter_td = alma_matter_header.find_next('td')
26                    alma_matter_list = alma_matter_td.find_next('> ul')
27                    if alma_matter_list:
28                        education_text = ', '.join(li.get_text(strip=True) for li in ul_element.find_all('li'))
29                        find_education=1
30
31                alma_matter_a = alma_matter_td.find_all('a')
32                if alma_matter_a and find_education==0 :
33                    education_text = ', '.join(a.get_text(strip=True) for a in alma_matter_td.find_all('a'))
34                    find_education=1
35
36            if find_education==1:
37                return education_text
38            else:
39                return 'NO_SAVE'
```

Έπειτα προχωρήσαμε στο να δημιουργήσουμε ένα dataset σε μορφή csv για να αποφύγουμε να επαναλαμβάνουμε την εκτέλεση του παραπάνω κώδικα μιας και ήταν αρκετά χρονοβόρος.

Into the CSV

```
1 sv_filename = 'scientists_data.csv'
2
3 # Headers
4 headers = ['Name', 'Awards', 'Education']
5
6 with open(csv_filename, 'w', newline='', encoding='utf-8') as csv_file:
7
8     csv_writer = csv.writer(csv_file)
9
10    # Write the headers
11    csv_writer.writerow(headers)
12
13    #write each scientist's information to the CSV file
14    for link in first_links_list:
15        name = get_scientist_name(link)
16        awards = get_scientist_awards(link)
17        education = get_scientist_education(link)
18
19
20    #Write the information
21    if education!='NO_SAVE' and not education.startswith('(') and not education.startswith('['):
22        csv_writer.writerow([name, awards, education])
23
```

Το επόμενο βήμα είναι να μετατρέψουμε το csv σε dataframe για την ευκολότερη διαχείριση των δεδομένων. Έπειτα δημιουργήσαμε μια λίστα με όλα τα εκπαιδευτικά ινστιτούτα που ανακτήθηκαν από τον κάθε computer scientist (ένας computer scientist μπορεί να έχει φοιτήσει σε παραπάνω από ένα ινστιτούτα) και αφαιρέσαμε τυχόν “σκουπίδια” που εμφανίστηκαν σε αυτή.

Clean Garbage

```
1 csv_file_path = 'scientists_data.csv'
2 df = pd.read_csv(csv_file_path)
3 all_institutes=[]
4 for row in df['Education']:
5     institutes = [inst.strip() for inst in row.split(',')]
6     all_institutes.extend(institutes)
7
8 unique_institutes = list(set(all_institutes))
9 unique_institutes.remove('Ph.D')
10 unique_institutes.remove('B.S.')
11 unique_institutes.remove('[11]')
12 unique_institutes.remove('[2]')
13 unique_institutes.remove('S.B.')
14 unique_institutes.remove('BSc.')
15 unique_institutes.remove('es')
16 unique_institutes.remove('BE')
17 unique_institutes.remove('[1]')
18 unique_institutes.remove('[3]')
19 unique_institutes.remove('[5]')
20 unique_institutes.remove('PhD')
21 unique_institutes.remove('[4]')
22 unique_institutes.remove('MSc')
23 unique_institutes.remove('B.Sc.')
24 unique_institutes.remove('BSc')
25 unique_institutes.remove('BASc')
26 unique_institutes.remove('Ph.D.')
27 unique_institutes.remove('1969')
28 unique_institutes.remove('MS')
29 unique_institutes.remove('M.S.')
30 unique_institutes.remove('SM')
31 unique_institutes.remove('BS')
32 unique_institutes.remove('SB')
33 unique_institutes.remove('BA')
34 unique_institutes.remove('AB')
35 unique_institutes.remove('B.A.')
36 unique_institutes.remove('Diplom')
37 unique_institutes.remove('')
```

Για την υλοποίηση αυτού του πρωτοκόλλου δημιουργήθηκαν δύο κλάσεις. Η πρώτη κλάση είναι των κόμβων. Έχει ως χαρακτηριστικά τον αριθμό του κόμβου, ένα fingertable με τους successors και τα πραγματικά indexes που έχει ο κάθε κόμβος. Τέλος έναν πίνακα αρχικά άδειο, ύστερα με τα κλειδιά του κόμβου καθώς και με τα values (δεδομένα) που αντιστοιχούν σε αυτόν τον κόμβο.

Node Class

```
1 class peer_node:
2     def __init__(self, df, node_num):
3         self.values=df
4         self.node_num=node_num
5         self.fingertable = pd.DataFrame(columns=['successors', 'index_of_peers'])
6         self.key=[]
7     def set_key(self, key):
8         self.key=key
9     def get_key(self, key):
10        self.key.append(key)
11    def merge_key_list(self, node2):
12        self.key=self.key+node2.key
13    def set_finger(self, succ, index):
14        self.fingertable['successors'] = succ
15        self.fingertable['index_of_peers'] = index
16
```

Ακολουθεί η κλάση του chord circle. Αρχικά όπως βλέπουμε τα ορίσματα της κλάσης είναι η λίστα με τα εκπαιδευτικά υδρίματα που δημιουργήθηκε νωρίτερα, το df που φτιάξαμε από το dataset, ο αριθμός του τελευταίου κόμβου, ένας πίνακας με όλους τους κόμβους και τέλος ένα dataframe που θα περιέχει για τον κάθε κόμβο το νούμερο του, σε ποιο κλειδί αντιστοιχεί καθώς και ποια κλειδιά πρέπει να αναζητήσουμε στον συγκεκριμένο κόμβο (αφού μπορεί να είναι παραπάνω από ένα). Έπειτα ακολουθούν ρουτίνες που χρησιμοποιούνται κατά την εκτέλεση των τριών βασικών λειτουργιών του αλγορίθμου. Έχουμε μια ρουτίνα την εκαθάριση των finger tables και μία για την δημιουργία των καινούριων. Εδώ πρέπει να αναφερθεί πως τα finger tables έχουν δύο τιμές για κάθε κόμβο, η πρώτη τιμή του είναι το node number και η δεύτερη το index στο chord circle για να μπορεί να εντοπισθεί κατευθείαν.

Chord Circle Class

```
1 class Chord_circle:
2     def __init__(self,institutes,df):
3         self.institutes=institutes
4         self.df=df
5         self.max_node=0
6         self.peers=[]
7         self.fake_hash_table=pd.DataFrame(columns=['number_of_key','key','search_key'])
8     def remove_finger_tables(self):
9         for peer in self.peers:
10             peer.fingertable=pd.DataFrame(columns=['successors'])
11     def create_finger_table_for_all(self):
12         length=len(self.peers)
13         m=math.ceil(math.log2(length))
14         temp_start=0
15         for peer in self.peers:
16             temp_table=[]
17             index_table=[]
18             first_attempt=0
19             for i in range(1,m+1):
20                 ok=0
21                 start=peer.node_num+2**(i-1)
22                 #if peer.node_num==405:
23                     #print("FINGERTABLE-->" + str(start)+ " WITH MAXNODE--> "+str(self.ma
24                 if start > self.max_node:
25                     start= start - self.max_node - 1
26                 if (self.return_to_start(peer.node_num)) & (first_attempt==0):
27                     #print("THIS IS NODE_NUM--> "+str(peer.node_num)) #DEBUGGING
28                     start=0
29                     first_attempt=1
30                 while(ok==0):
31                     for index,s_peer in enumerate(self.peers):
32                         if s_peer.node_num==start:
33                             temp_table.append(start)
34                             index_table.append(index)
35                             ok=1
36                     start+=1
37
38             peer.set_finger(temp_table,index_table)
```

Παρακάτω βλέπουμε την `renew_max_node` η οποία κρατάει το node number του τελευταίου κόμβου. Έπειτα είναι η συνάρτηση `create_list_of_peers` η οποία δημιουργεί τον chord circle. Κάθε institute στην λίστα-όρισμα εκχωρείται στον πίνακα `fake_hash_table` μαζί με τις υπόλοιπες τιμές που του αντιστοιχούν, δηλαδή το πραγματικό κλειδί, το κλειδί που έλλαβε για τις ανάγκες του αλγορίθμου και τον κόμβο στον οποίο βρίσκεται το κλειδί. Έπειτα δημιουργούμε έναν κόμβο για αυτό και τον προσθέτουμε στην λίστα με τους peers. Αφού ολοκληρωθεί αυτή η διαδικασία κάνουμε `renew` τον max node και εκτελούμε την function που δημιουργεί finger tables για όλους τους κόμβους. Τέλος έχουμε το function που ελέγχει αν είμαστε στον τελευταίο κόμβο και επιστρέφει 1 αν είμαστε αλλιώς 0.

```
def renew_max_node(self):
    self.max_node=int(self.peers[len(self.peers)-1].node_num)
    print("TYPE OF MAX NODE -->" +str(self.max_node))
    #print("NEW MAX NODE --> " +str(self.max_node)) DEBUGGING
def create_list_of_peers(self):
    counter=0

    for institute in self.institutes:
        result_rows = []
        self.fake_hash_table.loc[len(self.fake_hash_table)] = {'number_of_key': counter, 'key': institute, 'search_key': counter}

        for index, row in self.df.iterrows():

            if institute in row['Education']:
                result_rows.append(row.drop('Education'))

        result_df = pd.DataFrame(result_rows).reset_index(drop=True)
        result_df['institute'] = institute
        peer=peer_node(result_df,counter)
        counter+=1
        peer.get_key(institute)
        self.peers.append(peer)
    self.renew_max_node()
    self.create_finger_table_for_all()
def return_to_start(self,i):
    #print("MAX="+str(self.max_node)) DEBUGGING
    if (i != 0) & ((i % self.max_node) == 0):
        return 1
    else:
        return 0
```

Παρακάτω είναι το function που ελέγχει το fake hash table και ψάχνει το κλειδί του ιδρύματος που δίνουμε σαν όρισμα και επιστρέφει τον κόμβο στον οποίο βρίσκεται.

```
def key_to_number(self, lookup_key):
    get_num=None
    for index, row in self.fake_hash_table.iterrows():
        if row['key'].lower()==lookup_key.lower():
            get_num=row['search_key']
            #print('SEARCH KEY IS --->' +str(get_num)) DEBUGGING
            break
    if get_num==None:
        print("No archive for this Institute")
    return get_num
```

Παρακάτω βλέπουμε το `leave_circle` το οποίο ξεκινάει βρίσκοντας τον κόμβο που θέλουμε να διαγράψουμε. Έπειτα ελέγχουμε αν ο κόμβος αυτός υπάρχει, αν δεν υπάρχει σταματάμε. Αν υπάρχει ελέγχουμε το `fingertable` του για να βρούμε τον πρώτο `successor` του. Αφού βρούμε τον πρώτο διάδοχό του, του περνάμε τις τιμές και το κλειδί του κόμβου που θα διαγραφεί. Ενημερώνουμε το `fake hash table` για τις αλλαγές που έγιναν και τέλος διαγράφουμε τον κόμβο. Βρίσκουμε τον μέγιστο κόμβο(αν έχει αλλάξει), καθαρίζουμε τα `finger tables` και δημιουργούμε τα καινούρια.

```
def leave_circle(self,node):
    big=1
    for peer in self.peers:
        if peer.node_num==node:
            delete_this_node=peer
            big=0
    if big==1:
        print('There is no such node')
    elif node<=self.max_node:
        #if self.return_to_start(node):
        nn = delete_this_node.fingertable['index_of_peers'].iloc[0]
        print(f'NN is -->{nn}')
        next_node=self.peers[nn]
        next_node.values=pd.concat([next_node.values,delete_this_node.values],ignore_index=True)
        next_node.merge_key_list(delete_this_node)
        self.fake_hash_table.loc[self.fake_hash_table['search_key']==node, 'search_key']=next_node.node_num

        self.peers.remove(delete_this_node)
        self.renew_max_node()
        self.remove_finger_tables()
        self.create_finger_table_for_all()
```

Η συνάρτηση `show` δημιουργήθηκε για να έχουμε την δυνατότητα να δούμε ολόκληρο τον `chord circle`.

```
def show(self):
    for peer in self.peers:
        key_str = ', '.join(map(str, peer.key))
        print(str(peer.node_num) + ' --> ' + key_str)
```

Το lookup ξεκινάει βρίσκοντας το κλειδί του ινστιτούτου που αναζητήσαμε στο fake hash table. Αν βρεθεί αποτέλεσμα δημιουργούμε ένα dataframe για τα τελικά δεδομένα που πρέπει να εμφανίσει το query. Ξεκινάμε την αναζήτηση από το fingertable του πρώτου κόμβου του κύκλου. Υπάρχουν τρία ενδεχόμενα είτε βρήκαμε τον σωστό κόμβο στο fingertable ή ο κόμβος που ψάχνουμε είναι μεγαλύτερος από τον πιο μακρινό κόμβο που βλέπει το fingertable του κόμβου που βρισκόμαστε ή ο κόμβος που ψάχνουμε βρίσκεται ανάμεσα σε δύο successors. Η διαδικασία αναζήτησης επαναλαμβάνεται μέχρι να βρεθούμε στον κόμβο που αναζητάμε. Τέλος εμφανίζουμε τα δεδομένα που πληρούν τις προϋποθέσεις που θέσαμε.

```
def lookup(self, key, threshold):
    node_to_search=self.key_to_number(key)
    print(f"The key is --> {node_to_search}")
    #print('I WANT NODE'+str(node_to_search)) DEBUGGING
    if node_to_search!=None:
        res_df=pd.DataFrame()
        this_node=self.peers[0]
        while(this_node.node_num!=node_to_search):
            print(f"Searching {this_node.node_num} fingertable")
            ft=this_node.fingertable['successors']
            print(f"{ft}") #DEBUGGING
            for index,row in this_node.fingertable.iterrows():
                if row['successors']==node_to_search:
                    #print('1st if-->'+str(row['successors'])) #DEBUGGING
                    index=row['index_of_peers']
                    this_node=self.peers[index]
                    break

            elif row['successors']>node_to_search :
                #print('2nd if') #DEBUGGING
                this_node_index = index - 1
                if this_node_index >= 0:
                    previous_row = this_node.fingertable.at[this_node_index, 'index_of_peers']
                    print_row = this_node.fingertable.at[this_node_index, 'successors']
                    print(f'Previous_row = {print_row}')
                    this_node=self.peers[previous_row]
                    break
            else:
                #print('3rd if') #DEBUGGING
                if index == this_node.fingertable.index[-1]:
                    print_this=this_node.fingertable['successors'].iloc[-1]
                    print(f'Max row = {print_this}')
                    correct_index=this_node.fingertable['index_of_peers'].iloc[-1]
                    this_node=self.peers[correct_index]
                else: continue

        res_df=this_node.values
        filtered_rows = res_df[(res_df['Awards'] >= threshold) & (res_df['institute']==key)]
        print(filtered_rows)
```

Έπειτα έχουμε την συνάρτηση `join_circle` η οποία υλοποιεί το `join` του αλγορίθμου. Αρχικά ξεκινάμε βρίσκοντας τα κενά του κύκλου, αυτό προϋποθέτει ότι θα έχουμε διαγράψει κάποιους κόμβους ειδικά η μόνη κενή θέση είναι αυτή στο τέλος του κύκλου. Ιδανικά οι κόμβοι που προσθέτουμε είναι κόμβοι που προϋπήρχαν αφού τυχόν καινούριοι κόμβοι θα είναι άδεια. Επιπλέον να σημειωθεί πως έξτρα κόμβοι μπορούν να προστεθούν αφού βεβαιωθούμε πως ο τελευταίος κόμβος (406) υπάρχει ήδη στον κύκλο, εάν δεν υπάρχει δεν μπορούμε να προσθέσουμε κόμβο. Η παραδοχή αυτή έγινε με το σκεπτικό ότι εξαρχής δεν έχει νόημα η δημιουργία ενός κενού κόμβου και δημιουργήθηκε μόνο για τις ανάγκες της εγασίας.

```
def join_circle(self):
    empty_slots=[]
    last=0
    max_empty=self.fake_hash_table['number_of_key'].iloc[-1]
    stop=max_empty+1
    if max_empty==self.max_node:
        stop=stop+1
    for i in range(0,stop):
        if i>self.max_node:
            empty_slots.append(i)
        elif i<len(self.peers)-1:
            #print(self.peers[i].node_num) DEBUGGING
            print(f'Max node is {self.max_node} and i is {i+1}')
            D=self.peers[i+1].node_num-self.peers[i].node_num
            if D>1 :
                for j in range(self.peers[i].node_num+1,self.peers[i+1].node_num):
                    empty_slots.append(j)
    print("Choose a slot from these:")
    for slot in empty_slots:
        print(slot)
    sys.stdout.flush()
    chosen_node=int(input("Enter here ---> "))
```

Έπειτα ελέγχουμε την θέση την οποία επιλέξαμε. Αν υπάρχει αυτή η θέση είναι είτε η τελευταία είτε κάποια ενδιάμεση. Και στις δύο περιπτώσεις πρέπει να ελέγξουμε εάν ο επόμενος κόμβος περιέχει το κλειδί του κόμβου που κάνει `join` και αν όντως το έχει να γίνει η ανάθεση κλειδιού του τόσο στο node που δημιουργήθηκε όσο και στο fake hash table. Έπειτα διαχωρίζουμε τα values του κάθε κόμβου και ο κόμβος κάνει `join`. Τέλος ανανεώνουμε σε κάθε περίπτωση τον max node καθώς και τα finger tables των κόμβων.

```

if chosen_node not in empty_slots:
    print("Thats not a valid node!")
elif chosen_node==max(empty_slots) & self.fake_hash_table['number_of_key'].iloc[-1]!=chosen_node:
    empty_df=pd.DataFrame()
    peer_n=peer_node(empty_df,chosen_node)
    self.peers.append(peer_n)
else:
    new_keys=[]
    new_node_df=pd.DataFrame()
    pre=chosen_node-1
    while pre in empty_slots and pre>=0:
        pre=pre-1
    if chosen_node==max(empty_slots):
        next_node_num=self.peers[0].node_num
        last=1
    else:
        suc=chosen_node+1
        while suc in empty_slots:
            suc=suc+1
        next_node_num=suc
        while pre in empty_slots and pre>=0:
            pre=pre-1
            if pre<0:
                pre=self.max_node

    self.fake_hash_table.loc[(self.fake_hash_table['number_of_key'] <= chosen_node) & (self.fake_hash_table['number_of_key'] >pre), 'search_key'] = chosen_node
    condition=(self.fake_hash_table['search_key'] <= chosen_node) & (self.fake_hash_table['search_key'] >pre)
    selected_rows = self.fake_hash_table[condition]
    print(selected_rows)
    list_try=selected_rows['key'].tolist()
    for peer in self.peers:
        if peer.node_num==next_node_num:
            split_condition = peer.values['institute'].isin(list_try)
            new_node_df=peer.values[split_condition]
            peer.values=peer.values[~split_condition]
            peer_index=self.peers.index(peer)
            for key in list_try:
                new_keys.append(key)
                peer.key.remove(key)
            #print(peer.values)
    # self.peers(next_node).value
    peer_n=peer_node(new_node_df,chosen_node)
    peer_n.set_key(new_keys)
    if last==1:
        print(new_node_df)
        self.peers.append(peer_n)
    else:
        self.peers.insert(peer_index,peer_n)
self.renew_max_node()
self.remove_finger_tables()
self.create_finger_table_for_all()

```

ΠΕΙΡΑΜΑΤΙΚΗ ΑΞΙΟΛΟΓΗΣΗ

Ξεκινάμε με την δημιουργία αντικειμένου της κλάσης chord circle.

Δημιουργία αντικειμένου

Δημιουργία Chord Circle

```
1 Chord=Chord_circle(unique_institutes,df)
2 Chord.create_list_of_peers()
```

✓ 9.4s

Παρακάτω βλέπουμε τον αριθμό του κάθε κόμβου καθώς και το πανεπιστήμιο-κλειδί του.

Εμφάνιση κύκλου

```
1 Chord.show()
✓ 0.0s
```

```
0 --> Harvey Mudd College
1 --> Indian Institute of Technology
2 --> Brock University
3 --> Rhodes College
4 --> University of Alberta
5 --> Eötvös Loránd University
6 --> Dublin
7 --> University of Iowa
8 --> Sharif University of Technology
9 --> Wesleyan University
10 --> Indian Institute of Science
11 --> mathematics
12 --> Leningrad State University
13 --> Wayne State University
14 --> Trinity College
15 --> M.Eng.
16 --> Technical University of Denmark
17 --> American University
18 --> University at Buffalo
19 --> Eton College
20 --> National Technical University of Athens
21 --> Madras
22 --> Massachusetts Institute of Technology
23 --> University of Kansas City
24 --> Stanford University
...
403 --> Institute of Control Sciences
404 --> University of California - Berkeley
405 --> University of Lisbon
406 --> University of Cambridge
```

Εδώ βλέπουμε και το fake hash table που δημιουργήθηκε με την δημιουργία του αντικειμένου.

Εμφάνιση fake_hash_table

```
1 Fake_has_table=Chord.fake_hash_table
2 print(Fake_has_table)
```

✓ 0.0s

	number_of_key	key	search_key
0	0	Harvey Mudd College	0
1	1	Indian Institute of Technology	1
2	2	Brock University	2
3	3	Rhodes College	3
4	4	University of Alberta	4
..	...	...	...
402	402	Instituto Superior Técnico	402
403	403	Institute of Control Sciences	403
404	404	University of California - Berkeley	404
405	405	University of Lisbon	405
406	406	University of Cambridge	406

Θα ξεκινήσουμε διαγράφοντας μερικούς κόμβους έπειτα θα προσθέσουμε κάποιους άλλους και τέλος θα τρέξουμε μερικά queries.

Διαγραφή κόμβων:

Leave Circle κάποιων κόμβων

```
1 Chord.leave_circle(11)
2 Chord.leave_circle(12)
3 Chord.leave_circle(13)
4 Chord.leave_circle(311)
5 Chord.leave_circle(88)
6 Chord.leave_circle(173)
7 Chord.leave_circle(389)
8 Chord.leave_circle(404)
9 Chord.leave_circle(405)
10 Chord.leave_circle(406)
```

✓ 3.6s

Εδώ βλέπουμε τον chord circle μετά την διαγραφή των παραπάνω κόμβων. Όπως βλέπουμε τα κλειδιά των κόμβων 404, 405 και 406 πήγαν στον κόμβο 0 ενώ οι 11, 12 και 13 στον κόμβο 14.

Εμφάνιση κύκλου

```
1 Chord.show()
```

✓ 0.0s

```
0 --> Harvey Mudd College, University of Cambridge, University of Lisbon, University of California - Berkeley
1 --> Indian Institute of Technology
2 --> Brock University
3 --> Rhodes College
4 --> University of Alberta
5 --> Eötvös Loránd University
6 --> Dublin
7 --> University of Iowa
8 --> Sharif University of Technology
9 --> Wesleyan University
10 --> Indian Institute of Science
14 --> Trinity College, Wayne State University, Leningrad State University, mathematics
15 --> M.Eng.
16 --> Technical University of Denmark
17 --> American University
18 --> University at Buffalo
19 --> Eton College
20 --> National Technical University of Athens
21 --> Madras
22 --> Massachusetts Institute of Technology
23 --> University of Kansas City
24 --> Stanford University
25 --> New York University Tandon School of Engineering
26 --> College Park
27 --> Oberlin College
...
400 --> University of Hildesheim
401 --> Acadia University
402 --> Instituto Superior Técnico
403 --> Institute of Control Sciences
```

Παρατηρούμε τις αλλαγές και στον πίνακα με τα κλειδιά. Όπως φαίνεται τους κόμβους 404,405 και 406 για παράδειγμα θα τους αναζητήσουμε στον κόμβο 0.

Εμφάνιση fake_hash_table

```
1 Fake_has_table=Chord.fake_hash_table
2 print(Fake_has_table)
```

✓ 0.0s

	number_of_key	key	search_key
0	0	Harvey Mudd College	0
1	1	Indian Institute of Technology	1
2	2	Brock University	2
3	3	Rhodes College	3
4	4	University of Alberta	4
..	...	...	...
402	402	Instituto Superior Técnico	402
403	403	Institute of Control Sciences	403
404	404	University of California - Berkeley	0
405	405	University of Lisbon	0
406	406	University of Cambridge	0

Join κόμβων:

Παρακάτω φαίνονται οι κόμβοι που λείπουν, επιλέγουμε έναν από αυτούς και κάνει join στον κύκλο.

Join Circle κάποιων κόμβων

```
1 Chord.join_circle()
```

✓ 8.5s

Choose a slot from these:

11
12
13
88
173
311
389
404
405
406
12

	number_of_key	key	search_key
11	11	mathematics	12
12	12	Leningrad State University	12

Join Circle κάποιων κόμβων

```
1 Chord.join_circle()
✓ 6.3s
```

Choose a slot from these:

11
13
88
173
311
389
404
405
406
406

	number_of_key	key	search_key
404	404	University of California - Berkeley	406
405	405	University of Lisbon	406
406	406	University of Cambridge	406

Chord Circle μετά το Join:

Εμφάνιση κύκλου

```
1 Chord.show()
✓ 0.0s
```

0 --> Harvey Mudd College
1 --> Indian Institute of Technology
2 --> Brock University
3 --> Rhodes College
4 --> University of Alberta
5 --> Eötvös Loránd University
6 --> Dublin
7 --> University of Iowa
8 --> Sharif University of Technology
9 --> Wesleyan University
10 --> Indian Institute of Science
12 --> mathematics, Leningrad State University
14 --> Trinity College, Wayne State University
15 --> M.Eng.
16 --> Technical University of Denmark
17 --> American University
18 --> University at Buffalo
19 --> Eton College
20 --> National Technical University of Athens
21 --> Madras
22 --> Massachusetts Institute of Technology
23 --> University of Kansas City
24 --> Stanford University
25 --> New York University Tandon School of Engineering
26 --> College Park
...
401 --> Acadia University
402 --> Instituto Superior Técnico
403 --> Institute of Control Sciences
406 --> University of California - Berkeley, University of Lisbon, University of Cambridge

Εδώ βλέπουμε στο fake hash table πως τα κλειδιά που προηγουμένος ψάχναμε στον κόμβο 0 τώρα τα ψάχνουμε στον κόμβο 406 αφού αυτός είναι ο πιο κοντινός successor των κόμβων που βγήκαν.

Εμφάνιση fake_hash_table

```
1 Fake_has_table=Chord.fake_hash_table
2 print(Fake_has_table)
```

✓ 0.0s

	number_of_key	key	search_key
0	0	Harvey Mudd College	0
1	1	Indian Institute of Technology	1
2	2	Brock University	2
3	3	Rhodes College	3
4	4	University of Alberta	4
..	...	...	...
402	402	Instituto Superior Técnico	402
403	403	Institute of Control Sciences	403
404	404	University of California - Berkeley	406
405	405	University of Lisbon	406
406	406	University of Cambridge	406

Lookup:

Τα αποτελέσματα για το University of Cambridge χωρίς να μας νοιάζει πόσα βραβεία έχουν, ενώ στο ακριβώς επόμενο στιγμιότυπο βλέπουμε μόνο τους επιστήμονες με 4 ή παραπάνω βραβεία.

```
LookUps
```

```
1 Chord.lookup("University of Cambridge", 0)
```

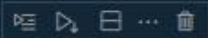
✓ 0.0s

```
2 392
3 396
4 406
5 14
6 45
7 109
8 237
```

Name: successors, dtype: int64

	Name	Awards	institute
1	Rediet Abebe	3	University of Cambridge
2	Rosemary Candlin	0	University of Cambridge
3	Christopher J. Date	0	University of Cambridge
4	Charlotte Froese Fischer	4	University of Cambridge
5	Geoffrey Hinton	9	University of Cambridge
6	Mathai Joseph	5	University of Cambridge
7	Derek McAuley	1	University of Cambridge
8	Alan Mycroft	0	University of Cambridge
9	Mike Paterson	10	University of Cambridge
10	Simon Peyton Jones	2	University of Cambridge
11	Cicely Popplewell	0	University of Cambridge
12	Martin Richards (computer scientist)	2	University of Cambridge
13	C. J. van Rijsbergen	0	University of Cambridge
14	Bjarne Stroustrup	11	University of Cambridge
15	Chai Keong Toh	3	University of Cambridge
16	Leslie Valiant	6	University of Cambridge
17	David Wheeler (computer scientist)	6	University of Cambridge
18	Yorick Wilks	19	University of Cambridge
19	Sophie Wilson	6	University of Cambridge
20	Neil Wiseman	0	University of Cambridge
21	Beatrice Worsley	0	University of Cambridge

LookUps



```
1 Chord.lookup("University of Cambridge",4)
```

✓ 0.0s

```
2 136
3 140
4 148
5 164
6 196
7 260
8 388
```

Name: successors, dtype: int64

Max row = 388

Searching 388 fingertable

```
0 390
1 390
2 392
3 396
4 406
5 14
6 45
7 109
8 237
```

Name: successors, dtype: int64

	Name	Awards	institute
4	Charlotte Froese Fischer	4	University of Cambridge
5	Geoffrey Hinton	9	University of Cambridge
6	Mathai Joseph	5	University of Cambridge
9	Mike Paterson	10	University of Cambridge
14	Bjarne Stroustrup	11	University of Cambridge
16	Leslie Valiant	6	University of Cambridge
17	David Wheeler (computer scientist)	6	University of Cambridge
18	Yorick Wilks	19	University of Cambridge
19	Sophie Wilson	6	University of Cambridge

Τέλος βλέπουμε την διαδικασία με την οποία πραγματοποιείται το lookup. Το κλειδί είναι το 406 και ξεκινώντας από τον κόμβο 0 ψάχνει στο fingertable του. Αφού δεν βρήκε το 406 πάει στο 256 που είναι ο πιο μακρινός κόμβος που βλέπει ο 0. Η διαδικασία συνεχίζεται με παρόμοιο τρόπο μέχρι που καταλήγει στον κόμβο 406.

```
The key is --> 406
Searching 0 fingertable
0      1
1      2
2      4
3      8
4     16
5     32
6     64
7    128
8    256
Name: successors, dtype: int64
Max row = 256
Searching 256 fingertable
0     257
1     258
2     260
3     264
4     272
5     288
6     320
7     384
8     105
Name: successors, dtype: int64
Max row = 384
```

```
Max row = 384
Searching 384 fingertable
0     385
1     386
2     388
3     392
4     400
5        9
6      41
7     105
8     233
Name: successors, dtype: int64
Max row = 400
Searching 400 fingertable
0     401
1     402
2     406
3        1
4        9
5       25
6       57
7     121
8     249
```